COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 6, 2026	**DATE SUBMITTED:**	October 12, 2026
NAME:	Reyes, Fenzie Aaron A Payas, John Paul S.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA – Measure the Runtime of Algorithm Using Theoretical Analysis

CODE

main.cpp > main()

```cpp
#include <iostream>
using namespace std;

// Search for a value in an array
int search(int x[], int m, int value) {
    for (int i = 0; i < m; i++){
        if (x[i] == value){
            return i;
        }
    }
    return -1;
}

// Check if the two arrays have any common values
bool diff(int x[], int m, int y[], int n){
    for (int i = 0; i < n; i++){
        if (search(x, m, y[i]) != -1){
            return false;
        }
    }
    return true;
}
```

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

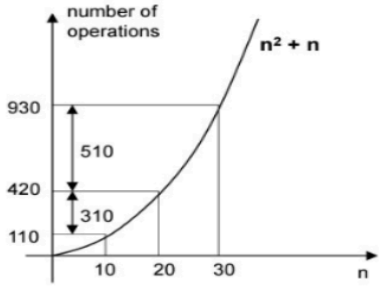
Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

CODE OUTPUT

```
24  int main() {
25      int x[] = {1, 2, 3, 4, 5};
26      int y[] = {6, 7, 8, 9, 10};
27
28      int m = sizeof(x) / sizeof(x[3]);
29      int n = sizeof(y) / sizeof(x[0]);
30
31      if (diff(x, m, y, n)) {
32          cout << "No common values found in the array." << endl;
33      } else {
34          cout << "Common value found in the array." << endl;
35      }
36
37      return 0;
38  }
```

FROM THE GRAPH AND TABLE:



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS TO THE WORKSHEET:

The given graph shows that as the number of operations increases, the n / array size increases as well. The worst-case runtime in this graph is $T(n) = n^2+n$, which represents a quadratic growth. In other words, as the size of the array increases, the number of operations increases significantly. Therefore, the algorithm has a worst-case time complexity of $O(n^2)$.

ILOB – Analyzing the Results of Runtime Performance by Comparing Experimental and Theoretical Analysis

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student{
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    //constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
    }

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator ==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator< (const Student& p) const {
        return this->name < p.name;
    }

    bool operator> (const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type>
        uniform_dist(1, max);

    // Generate random credentials
    auto random_name =
        std::make_pair(uniform_dist(rand), uniform_dist(rand));
```

```
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;

    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (first != last)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            last = first + mid_element_index;
        else
            first = first + mid_element_index + 1;
    }

    // Student not found in the sequence and therefore should be vaccinated
    return true;
}

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);
```

```
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();

    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " "
        << p.get_name().second
        << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " "
        << p.get_name().second
        << ") " << "does not need vaccination." << std::endl;
}

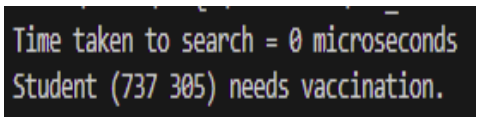
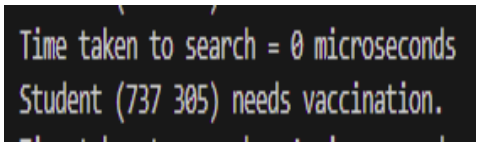
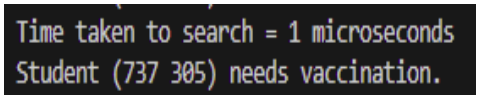
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);

    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

CODE OUTPUT

```
Time taken to search = 0 microseconds
Student (737 305) needs vaccination.
Time taken to search = 0 microseconds
Student (737 305) needs vaccination.
Time taken to search = 1 microseconds
Student (737 305) needs vaccination.
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	fast		The search is completed quickly because the input size is small.
10,000	fast		The execution time increases slightly as the input size increases.
100,000	fast		The search still performs efficiently because binary search is used.

CODE ANALYSIS:

The results show that the search time increases slightly as the number of students increases. As the input size grows, the program remains efficient because it uses binary search to find the student. Since binary search checks the middle of the list and reduces the search area by half each time, it can handle larger inputs quickly. As a result, the difference in search time remains small even with 100,000 students. Overall, these results support the O(log N) time complexity of the algorithm.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC – Theoretical Analysis, Experimental Analysis, and Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Reyes' Analysis

Theoretical Analysis

- The algorithm uses two nested loops to compare each element with the other elements in the array. If two elements are equal, it returns false; otherwise, it returns true after checking all pairs.
  Best Case: O(1) – If a duplicate is found in the first comparison.
  Worst Case: $O(n^2)$ – If all elements are unique or the duplicate is found at the last comparison.
  Space Complexity: O(1) – The algorithm uses only a fixed amount of extra memory.

Experimental Analysis

- Similarly, for the experimental analysis, the algorithm can also be tested using the same formula in the previous theoretical analysis. For instance, we have n that is equal to 5, inputting this into the formula will result in 10 comparisons. So the relationship is still directly proportional to each other because as the input increases, the number of comparisons and actual runtime also increase, especially when we deal with unique elements because the algorithm must first complete all possible comparisons before returning true based on the given algorithm shown.

Problem1.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <chrono>
using namespace std;
using namespace chrono;

bool Unique(int A[], int n, long long& operations) {
    operations = 0;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            operations++;

            if (A[i] == A[j])
                return false;
        }
    }

    return true;
}
```

```cpp
int main() {
    int sizes[] = {1000, 2000, 4000, 8000, 16000};

    cout << left
         << setw(15) << "N"
         << setw(25) << "Operations"
         << "Time (ms)\n";

    cout << string(55, '-') << "\n";

    for (int s = 0; s < 5; s++) {
        int n = sizes[s];
        int* A = new int[n];

        for (int i = 0; i < n; i++)
            A[i] = i + 1;

        long long operations = 0;

        auto start = steady_clock::now();
        Unique(A, n, operations);
        auto end = steady_clock::now();

        double time = duration<double, milli>(
            end - start).count();

        cout << left
             << setw(15) << n
             << setw(25) << operations
             << fixed << setprecision(4)
             << time << " ms\n";

        delete[] A;
    }

    return 0;
}
```

```
Reyes\OneDrive\Documents\todei\"Problem1
N               Operations                Time (ms)
-------------------------------------------------------
1000            499500                    1.7234 ms
2000            1999000                   7.5409 ms
4000            7998000                   33.7353 ms
8000            31996000                  126.3234 ms
16000           127992000                 458.0795 ms

[Done] exited with code=0 in 2.327 seconds
```

	Analysis and Comparison - The experimental results are expected to be consistent with the theoretical analysis because the runtime increases rapidly as the input size grows. Since the algorithm performs a large number of comparisons for larger arrays, its performance follows a quadratic growth pattern. Therefore, the experimental results support the theoretical time complexity of $O(n^2)$
Payas' Analysis	**Theoretical Analysis** - The best-case time complexity is O(1) when a duplicate is found immediately. The worst-case time complexity is $O(n^2)$ because the algorithm may compare every pair of elements. **Experimental Analysis** - The execution time was measured using different input sizes. As the input size increases, the execution time generally increases because more elements need to be compared.

```cpp
#include <iostream>
#include <chrono>

bool Unique(const int A[], int n, long long& op_count) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            op_count++;
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

int main() {
    int sizes[] = {1000, 2000, 4000, 8000, 16000};

    for (int n : sizes) {
        int* A = new int[n];
        for (int i = 0; i < n; i++) {
            A[i] = i;
        }
        long long op_count = 0;
        auto start = std::chrono::steady_clock::now();
        bool result = Unique(A, n, op_count);
        auto stop = std::chrono::steady_clock::now();
        auto elapsed =
            std::chrono::duration_cast<std::chrono::microseconds >(stop - start).count();

        std::cout << "N = " << n << "\tOperations = " << op_count << "\tTime = " << elapsed << " us\n";
        delete[] A;
    }

    return 0;
}
```

```
N = 1000    Operations = 499500     Time = 850 us
N = 2000    Operations = 1999000    Time = 3316 us
N = 4000    Operations = 7998000    Time = 13626 us
N = 8000    Operations = 31996000   Time = 58860 us
N = 16000   Operations = 127992000  Time = 231695 us
```

Analysis and Comparison

- The experimental results support the theoretical analysis. The algorithm becomes slower as the input size increases, especially in the worst case where its time complexity is $O(n^2)$.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Reyes' Analysis

(STILL IN PROGRESS)

Theoretical Analysis

- The rpower algorithm uses recursion to multiply \(x\) by itself \(n\) times. The brpower algorithm uses a faster approach by dividing the exponent in half during each recursive call.

Algorithm 1: rpower
Best Case: O(1) – When n = 0, it immediately returns 1.
Worst Case: O(n) – It makes one recursive call for each decrease in the exponent.
Space Complexity: O(n) – Recursive calls use additional stack memory.

Algorithm 2: brpower
Best Case: O(1) – When n = 0, it immediately returns 1.
Worst Case: O(log n) – The exponent is divided approximately in half during each recursive call.
Space Complexity: O(log n) – It requires fewer recursive calls than rpower.

Experimental Analysis

- The experimental results show that rpower makes more recursive calls as the exponent increases, from 101 calls at N = 100 to 1,601 calls at N = 1600. Its execution time also generally increases. In contrast, brpower only increases from 8 to 12 recursive calls, while its execution time remains almost constant. These results support the theoretical analysis that rpower has O(n) time complexity, while brpower has O(log n), making brpower more efficient for larger exponents.

Problem2REYES.cpp > ...

```cpp
#include <iostream>
#include <iomanip>
#include <chrono>
using namespace std;
using namespace chrono;

long long rOperations, brOperations;

long long rpower(long long x, int n) {
    rOperations++;
    if (n == 0)
        return 1;
    return x * rpower(x, n - 1);
}

long long brpower(long long x, int n) {
    brOperations++;
    if (n == 0)
        return 1;
    long long y;
    if (n % 2 == 1) {
        y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }
    y = brpower(x, n / 2);
    return y * y;
}
```

```cpp
int main() {
    int sizes[] = {100, 200, 400, 800, 1600};

    cout << left
         << setw(10) << "N"
         << setw(20) << "rpower Calls"
         << setw(20) << "rpower (ms)"
         << setw(20) << "brpower Calls"
         << "brpower (ms)\n";

    cout << string(90, '-') << "\n";
```

```
42      for (int i = 0; i < 5; i++) {
43          int n = sizes[i];
44          rOperations = 0;
45          auto start1 = steady_clock::now();
46          rpower(1, n);
47          auto end1 = steady_clock::now();
48          double rTime = duration<double, milli>(
49              end1 - start1).count();
50          brOperations = 0;
51          auto start2 = steady_clock::now();
52          brpower(1, n);
53          auto end2 = steady_clock::now();
54
55          double brTime = duration<double, milli>(
56              end2 - start2).count();
57
58          cout << left
59               << setw(10) << n
60               << setw(20) << rOperations
61               << setw(20) << fixed << setprecision(4) << rTime
62               << setw(20) << brOperations
63               << brTime << "\n";
64      }
65
66      return 0;
67  }
```

Analysis and Comparison

- The theoretical analysis shows that rpower has a time complexity of O(n), while brpower has a time complexity of O(log n). In the experimental results, rpower should make more recursive calls as the exponent increases, while brpower requires far fewer calls because it divides the exponent in half. Therefore, brpower is generally more efficient for large exponents, both in execution time and recursive stack usage.

Payas' Analysis

Theoretical Analysis

- For rpower(x,n), the time complexity is O(n) because the exponent decreases by 1 for every recursive call. For brpower(x,n), the time complexity is O(log n) because the exponent is divided by 2 in each recursive call.

Experimental Analysis

- The execution time of both power functions was measured using different values of n. As n increases, rpower() requires more recursive calls, while brpower() requires significantly fewer calls.

```cpp
#include <iostream>
#include <chrono>

long long rpower(long long x, int n, long long& ops) {
    if (n == 0) return 1;
    ops++;
    return x * rpower(x, n - 1, ops);
}
long long brpower(long long x, int n, long long& ops) {
    if (n == 0) return 1;
    long long y = brpower(x, n / 2, ops);
    if (n % 2 == 1) {
        ops += 2;
        return x * y * y;
    }
    ops++;
    return y * y;
}

int main() {
    int sizes[] = {100, 500, 1000, 2000, 4000};

    for (int n : sizes) {
        long long a = 0, b = 0;

        auto start = std::chrono::steady_clock::now();
        rpower(1, n, a);
        auto end = std::chrono::steady_clock::now();
        auto t1 = std::chrono::duration_cast<
            std::chrono::microseconds>(end - start).count();

        start = std::chrono::steady_clock::now();
        brpower(1, n, b);
        end = std::chrono::steady_clock::now();
        auto t2 = std::chrono::duration_cast<
            std::chrono::microseconds>(end - start).count();

        std::cout << "N = " << n << "  rpower Ops = " << a << "  Time = " << t1 << " us\n";
        std::cout << "N = " << n << "  brpower Ops = " << b << "  Time = " << t2 << " us\n";
    }

    return 0;
}
```

```
N = 100  rpower Ops = 100  Time = 0 us
N = 100  brpower Ops = 10  Time = 0 us
N = 500  rpower Ops = 500  Time = 29 us
N = 500  brpower Ops = 15  Time = 0 us
N = 1000  rpower Ops = 1000  Time = 33 us
N = 1000  brpower Ops = 16  Time = 0 us
N = 2000  rpower Ops = 2000  Time = 63 us
N = 2000  brpower Ops = 17  Time = 0 us
N = 4000  rpower Ops = 4000  Time = 147 us
N = 4000  brpower Ops = 18  Time = 0 us
```

Analysis and Comparison

- The experimental results show that brpower(x,n) is more efficient than rpower(x,n) for larger values of n. This agrees with the theoretical complexities of O(log n) and O(n), respectively.

CONCLUSION – PAYAS

This activity helped us understand the importance of theoretical and experimental analysis in evaluating algorithm performance. We learned how time complexity affects execution time as the input size increases, particularly in algorithms with $O(n^2)$, $O(n)$, and $O(\log n)$ time complexities. By comparing the results, we found that algorithms such as binary search and binary recursive power are more efficient for handling larger inputs. This activity improved our understanding of algorithm efficiency and helped us recognize the importance of choosing appropriate algorithms for better performance.

CONCLUSION – REYES

This activity helped me understand the importance of theoretical and experimental analysis in evaluating the efficiency of algorithms. The series of activities within this activity taught how time complexity affects an algorithm's performance as the input size increases, particularly in algorithms with $O(n^2)$, $O(n)$, and $O(\log n)$ time complexities. Through the experiments, I observed that algorithms with quadratic time complexity become less efficient as the input size grows, while algorithms such as binary search and binary recursive power can handle larger inputs more efficiently. I also learned that theoretical analysis provides an estimate of an algorithm's performance, while experimental analysis allows us to observe its actual behavior. Overall, this activity improved my understanding of algorithm efficiency and the importance of choosing appropriate algorithms to achieve better performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance and Explanation of Concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only paved the way for us to help debug and optimize our code for a better output. It also helped us understand the concept of measuring time, time complexity cases, and exploring effects on performance. Other than that, it did not replace our original idea or the desired outcome we wanted to obtain.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.